✅ Final Comprehensive List (duplicates removed, logically ordered)

1. `StringBuilder.reverse()` – Recommended

```
String reversed = new StringBuilder(original).reverse().toString();
```

- Fast, clean, the obvious first choice.

2. `StringBuffer.reverse()` – Thread-safe variant

```
String reversed = new StringBuffer(original).reverse().toString();
```

- Use only when thread safety is needed (rare).

3. Two-pointer swap on `char[]` – In-place efficient

```
char[] chars = original.toCharArray();
int left = 0, right = chars.length - 1;
while (left < right) {
    char tmp = chars[left];
    chars[left++] = chars[right];
    chars[right--] = tmp;
}
String reversed = new String(chars);
```

4. Build `char[]` from the end – Simple manual reverse

```
char[] result = new char[original.length()];
for (int i = 0; i < result.length; i++) {
    result[i] = original.charAt(original.length() - 1 - i);
}
String reversed = new String(result);
```

5. Naïve `for` loop with `+=` concatenation – Inefficient, avoid

```
String reversed = "";
for (int i = original.length() - 1; i >= 0; i--) {
    reversed += original.charAt(i);
}
```

6. `substring()` concatenation – Similar poor performance

```
String reversed = "";
for (int i = original.length(); i > 0; i--) {
    reversed += original.substring(i - 1, i);
}
```

7. `StringBuilder.insert(0, char)` – Prepending ($O(n^2)$)

```
StringBuilder sb = new StringBuilder();
for (char c : original.toCharArray()) {
    sb.insert(0, c);
}
String reversed = sb.toString();
```

8. Recursion (head recursion, quadratic time)

```
public static String reverseRecursive(String s) {
    if (s.isEmpty()) return s;
    return reverseRecursive(s.substring(1)) + s.charAt(0);
}
```

9. Tail-recursion helper

```
public static String reverseTail(String s) {
    return reverseTail(s, "");
}
private static String reverseTail(String rem, String acc) {
    if (rem.isEmpty()) return acc;
    return reverseTail(rem.substring(1), rem.charAt(0) + acc);
}
```

10. `Stack` / `Deque` (LIFO)

```
Deque<Character> stack = new ArrayDeque<>();
for (char c : original.toCharArray()) stack.push(c);
StringBuilder sb = new StringBuilder();
while (!stack.isEmpty()) sb.append(stack.pop());
String reversed = sb.toString();
```

11. `Collections.reverse()` on a `List<Character>`

```
List<Character> list = new ArrayList<>();
for (char c : original.toCharArray()) list.add(c);
Collections.reverse(list);
StringBuilder sb = new StringBuilder();
list.forEach(sb::append);
String reversed = sb.toString();
```

12. Java 8 Stream – `IntStream.range()` collecting into `StringBuilder`

```
String reversed = IntStream.range(0, original.length())
    .map(i -> original.charAt(original.length() - 1 - i))
    .collect(StringBuilder::new,
             StringBuilder::appendCodePoint,
             StringBuilder::append)
    .toString();
```

13. Stream `reduce()` – Concatenation (quadratic)

```
String reversed = original.chars()
    .mapToObj(c -> String.valueOf((char) c))
    .reduce("", (a, b) -> b + a);
```

14. Stream `collect()` with prepend (`sb.insert(0, c)`)

```
String reversed = original.chars()
    .mapToObj(c -> (char) c)
    .collect(Collector.of(
        StringBuilder::new,
        (sb, ch) -> sb.insert(0, ch),
        (sb1, sb2) -> sb1.insert(0, sb2),
        StringBuilder::toString));
```

15. Parallel stream (order not guaranteed, overkill)

```
String reversed = original.chars()
    .parallel()
    .mapToObj(c -> String.valueOf((char) c))
    .reduce("", (a, b) -> b + a);
```

16. `StringJoiner` + reduce (contrived)

```
String reversed = original.chars()
```

```
    .mapToObj(c -> String.valueOf((char) c))
    .reduce(new StringJoiner(""),
            (sj, s) -> sj.add(s),
            StringJoiner::merge)
    .toString();
// Note: order depends on implementation; not reliable.
```

17. Lambda / `Function` wrapper

```
Function<String, String> reverse = s -> new StringBuilder(s).reverse().toString();
String result = reverse.apply(original);
```

18. Xor-swap on `char[]` (gimmick)

```
char[] chars = original.toCharArray();
int left = 0, right = chars.length - 1;
while (left < right) {
    chars[left] ^= chars[right];
    chars[right] ^= chars[left];
    chars[left] ^= chars[right];
    left++;
    right--;
}
String reversed = new String(chars);
```

19. Unicode-safe reversal using `codePoints()` – 🌍 Handles emoji / supplementary characters

```
int[] codePoints = original.codePoints().toArray();
int left = 0, right = codePoints.length - 1;
while (left < right) {
    int tmp = codePoints[left];
    codePoints[left++] = codePoints[right];
    codePoints[right--] = tmp;
}
String reversed = new String(codePoints, 0, codePoints.length);
```

🔄 Variants (Beyond basic character reversal)

🚀 Reverse words (preserving word order, reversing each word)

```
String sentence = "Hello World";
String reversedWords = Pattern.compile(" ")
```

```
String reversedWords = Pattern.compile(" ")
    .splitAsStream(sentence)
    .map(w -> new StringBuilder(w).reverse())
    .collect(Collectors.joining(" "));
// "olleH dlroW"
```

🌍 Unicode-safe reversal – Already method #19 above.

Always use `codePoints()` when strings may contain emoji, accented characters, or characters outside the Basic Multilingual Plane (e.g., `"😊👍"`). A naive `char`-based reversal will corrupt surrogate pairs.

⚡ Benchmark insight (typical relative performance)

Method	Speed	Notes
`StringBuilder.reverse()`	⚡⚡⚡⚡⚡	Fastest, native optimisations
`char[]` two-pointer swap	⚡⚡⚡⚡	Close second, no extra buffer
Streams (`collect` into `StringBuilder` with code point mapping)	⚡⚡⚡	Good, but overhead from boxing
Streams with `reduce` / `insert(0, …)`	⚡	$O(n^2)$ – avoid for large strings
Recursion / `Stack` / `Collections.reverse`	⚡	Higher memory/time due to object allocation

---

All examples are pure Java, no third-party libraries. Use **#1** for production, **#19** for robustness, and the others for learning or interviews.